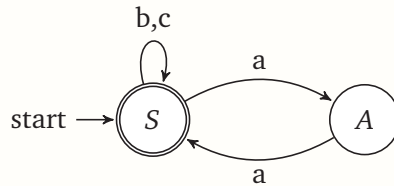


1. For each scenario below, construct (or give a formal outline of) a nondeterministic finite-state automata that accepts the described set of strings. Keep the number of nodes in your machine minimal. If useful, include a brief rationale for your construction.

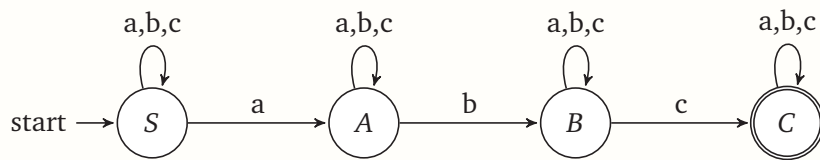
- (a) All strings over  $\{a, b, c\}^*$  in which every nonempty maximal substring of consecutive  $a$ 's is of even length.

**Solution:**



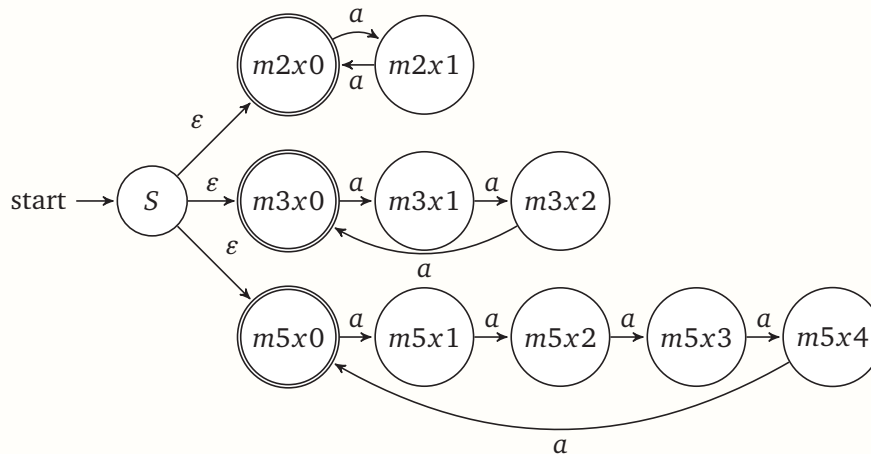
- (b)  $\Sigma^* a \Sigma^* b \Sigma^* c \Sigma^*$

**Solution:**



- (c) All strings in  $w \in a^*$  of length that is divisible by at least one of the following numbers 2, 3, 5. For full credit your automata should have less than (say) 15 states.

**Solution:**



- (d) All strings in  $w \in a^*$  of length that is **NOT** divisible by at least one of the following numbers 2, 3, 5.

**Solution:** Flipping the accept / reject state of the NFA in part (c) except the initial state would give the NFA for this problem. With  $M = (\Sigma, Q, s, A, \delta)$  as the NFA for part (c), we define the NFA  $M' = (\Sigma, Q', s', A', \delta')$  for part (d) as the following:

$$Q' = Q$$

$$s' = s$$

$$A' = Q \setminus (A \cup \{s\})$$

$$\delta'(q, \sigma) = \delta(q, \sigma) \quad \forall q \in Q, \sigma \in \Sigma$$

■

2. Suppose you are given a deterministic machine  $M = (\Sigma, Q, \delta, s, A)$  that recognizes some language  $L$ . Your task is to design a nondeterministic counterpart  $N$  that accepts a modified version of this language in each case below. Include a concise justification for your approach where relevant.

(a)  $\text{RemoveOnes}(L) := \{0^{\#_0(w)} \mid w \in L\}$ ; i.e., removes all 1s from the strings.

**Solution:** The main idea is that we use epsilon transitions to emulate processing 1 symbols even without reading them. As such, the new is defined as follows:

States:  $Q' = Q$ .

Start state:  $s' = s$ .

Accept states:  $A' = A$ .

Transition function:

$$\delta'(q, c) = \begin{cases} \{\delta(q, 0)\} & c = 0 \\ \emptyset & c = 1 \\ \{\delta(q, 1)\} & c = \epsilon \end{cases}$$

Any walk accepting  $w$  in the original DFA corresponds to a walk that accepts just the 0s in  $w$  in this NFA. ■

(b)  $\text{RemoveOnes}^{-1}(L) := \{w \in \Sigma^* \mid 0^{\#_0(w)} \in L\}$ ; i.e., puts back the 1s.

**Solution:** We simulate the DFA  $M$  on the projection of the string that keeps only the 0s. To do this, we ignore all 1s during simulation and only move on 0s. Formally:

States:  $Q' = Q$ .

Start state:  $s' = s$ .

Accept states:  $A' = A$ .

Transition function:

$$\delta'(q, c) = \begin{cases} \{\delta(q, 0)\} & c = 0 \\ \{q\} & c = 1 \end{cases}$$

The 1s act as no-ops; only 0s matter for accepting. ■

### 3. Fooling Sets

Prove that the following languages are not regular by providing a fooling set. You need to provide an infinite set and also prove that it is a valid fooling set for the given language.

(a)  $L = \{ww^Rw \mid w \in \{0,1\}^*\}$

**Solution:** An infinite fooling set is  $\{0^i110^i\}$ . Given  $x = 0^i110^i$  and  $y = 0^j110^j$  where  $i < j$ , distinguish with suffix  $z = 0^i1$ .

Then  $xz$  is in the language by construction (can be split into  $ww^Rw$ ), but  $yz$  is not: the structure of  $yz$  breaks since the first third would have 1s where only 0s should appear.

Thus,  $xz \in L$  but  $yz \notin L$ , proving this is a valid fooling set. ■

(b)  $L = \{0^i10^j \mid i \text{ is divisible by } j\}$

**Solution:** Let  $F = \{0^p1 \mid p > 0\}$ . For any distinct  $x = 0^m1$  and  $y = 0^n1$  in  $F$ , choose  $z = 0^m$ .

Then  $xz = 0^m10^m$  is in  $L$  (since  $m$  is divisible by  $m$ ), but  $yz = 0^n10^m$  is not necessarily in  $L$  if  $n$  is not divisible by  $m$ .

This proves the set  $F$  satisfies the fooling set conditions. ■

(c)  $L = \{a^i b^j \mid j = \log_2 i\}$

**Solution:** Let  $F = \{a^{2^k} b^k \mid k \geq 0\}$ . This is infinite since  $k$  ranges over all non-negative integers.

For  $x = a^{2^i} b^i$  and  $y = a^{2^j} b^j$  with  $i \neq j$ , the string  $x$  concatenated with  $b^{j-i}$  breaks the logarithmic condition for  $i \neq j$ .

Thus,  $F$  is a fooling set for the language. ■

(d)  $L = \{0^i 0^j \mid j = \sqrt{i}\}$

**Solution:** Let  $F = \{0^{k^2} 0^k \mid k \in \mathbb{N}\}$ . Any two elements  $x = 0^{k^2} 0^k$  and  $y = 0^{m^2} 0^m$  where  $k \neq m$  can be distinguished using the suffix  $0^{k-m}$  or a prefix analysis on square root conditions.

Since only  $k$  satisfies  $\sqrt{k^2} = k$ , any mismatch violates the language condition.

Hence  $F$  is a valid fooling set. ■

(e)  $L = \{wcd\#a(w) \mid w \in \{a, b\}^*\}$

**Solution:** Let  $F = \{a^kcd\#a^k \mid k \geq 0\}$ . Each element represents a string where the number of  $a$ 's in  $w$  equals the number of  $a$ 's after the “.

If  $x = a^icd\#a^i$  and  $y = a^jcd\#a^j$  with  $i \neq j$ , they can be distinguished by suffixes like ‘ $cd\#a^i$ ’ which matches  $x$  but not  $y$ .

Thus  $F$  satisfies the fooling set criteria. ■